Report

v. 2.0

Customer

Enso



Smart Contract Audit

Weiroll V2

20th June 2025

Report prepared by

ABDK Consulting

Contents

1 Changelog **4**

2 Introduction **5**

3 Project scope **6**

4 Methodology **7**

5 Our findings **8**

6 Major Issues **9**
CVF-2. FIXED 9
CVF-4. FIXED 9

7 Moderate Issues **10**
CVF-1. FIXED 10
CVF-3. FIXED 10
CVF-5. INFO 11
CVF-6. FIXED 11
CVF-7. FIXED 11
CVF-8. FIXED 12
CVF-9. INFO 12
CVF-11. INFO 13

8 Recommendations **14**
CVF-12. INFO 14
CVF-13. INFO 14
CVF-14. INFO 14
CVF-15. INFO 15
CVF-16. INFO 15
CVF-17. INFO 15
CVF-18. INFO 16
CVF-19. INFO 16
CVF-20. FIXED 16
CVF-21. FIXED 16
CVF-22. INFO 17
CVF-23. FIXED 17
CVF-24. INFO 17
CVF-25. INFO 18
CVF-26. INFO 18
CVF-27. INFO 19
CVF-28. INFO 19
CVF-29. INFO 20
CVF-30. INFO 20

CVF-31. INFO 21
CVF-32. FIXED 21
CVF-33. INFO 21
CVF-34. INFO 22
CVF-35. FIXED 22
CVF-36. INFO 22
CVF-37. FIXED 23
CVF-38. INFO 23
CVF-41. INFO 24
CVF-42. INFO 24
CVF-43. INFO 25
CVF-44. INFO 25

1 Changelog

#	Date	Author	Description
0.1	16.06.25	A. Zveryanskaya	Initial Draft
0.2	16.06.25	A. Zveryanskaya	Minor revision
1.0	16.06.25	A. Zveryanskaya	Release
1.1	20.06.25	A. Zveryanskaya	Removed: CVF-10,CVF-40
2.0	20.06.25	A. Zveryanskaya	Release

2 Introduction

All modifications to this document are prohibited. Violators will be prosecuted to the full extent of the U.S. law.

The following document provides the result of the audit performed by ABDK Consulting (Mikhail Vladimirov and Dmitry Khovratovich) at the customer request. The audit goal is a general review of the smart contracts structure, critical/major bugs detection and issuing the general recommendations.

Weiroll is a simple and efficient operation-chaining/scripting language for the EVM.

3 Project scope

We were asked to review:

- Original Code
- Code with Fixes

Files:

```
/
```

cdptr.sol	constants.sol	dynmem.sol
IWETH.sol	WeirollErrors.sol	WeirollVM.sol

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment**. The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis**. Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis**. For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis**. The code logic of particular functions is analysed for correctness and efficiency. We check if code actually does what it is supposed to do, if that algorithms are optimal and correct, and if proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** contain code style, best practices and other suggestions.

5 Our findings

We found 2 major, and a few less important issues. All identified Major issues have been fixed.



Fixed 2 out of 2 issues

6 Major Issues

CVF-2 FIXED

- **Category** Unclear behavior
- **Source** WeirollVM.sol

Description This allows bypassing the “within_bounds” check, so “op” could be read outside of “full_instructions”.

Recommendation Do “within_bounds” check at the beginning of the loop body, right after reading “op”.

118
```
break;
```

CVF-4 FIXED

- **Category** Flaw
- **Source** WeirollVM.sol

Description This allows reading arbitrary regions of EVM memory.

Recommendation Restrict memory access or clearly explain why reading arbitrary regions of memory is fine.

Client Comment *This can be an unsafe operation, it depends on the program being processed. It was determined that for efficiency’s sake it should be up to the program creator to ensure memory safety such that non-ptr registers are not treated as ptr registers. Documented in commit 1762c98.*

594
```
    dynmem value = dynmem_lib.unsafe_from_word(word);
```

615
```
value = dynmem_lib.unsafe_from_word(self.get_reg(input_byte.reg_idx
    ↪ ()));
```

7 Moderate Issues

CVF-1 FIXED

- **Category** Overflow/Underflow
- **Source** dynmem.sol

Description Overflow is possible here.

Recommendation Use safe addition.

Client Comment *Client deems severity overstated as no concrete impact has been demonstrated. While use of function may result in overflow its use in the VM does not. Function renamed and documentation added to address concerns regarding safety and adherence to codebase conventions.*

```
22 mstore(0x40, add(new_mem_addr, length))
```

CVF-3 FIXED

- **Category** Unclear behavior
- **Source** WeirollVM.sol

Description ABI encoding assumes length to be stored inside returndata, rather then derived from returndatasize.

Recommendation Read the length from returndata or clearly explain what encoding is used here.

Client Comment *Functions as expected, please reference Weiroll V1. But essentially it partially decodes an abi encoded '(dyn,)' value which will always start with 0x20, the remaining bytes are stored. Readme updated to make this slightly clearer.*

```
456 retdata_length := sub(returndatasize(), 0x20)
```

CVF-5 INFO

- **Category** Procedural
- **Source** dynmem.sol

Description Solidity documentation recommends to pad free memory pointer to 32 bytes.

Recommendation Do the padding.

Client Comment *Solidity recommends but does not require, none of the code expects the fmp to be aligned.*

```
22 mstore(0x40, add(new_mem_addr, length))
```

CVF-6 FIXED

- **Category** Procedural
- **Source** dynmem.sol

Description In case “self.len()” is less than 32, this will revert without an error message, due to checked underflow.

Recommendation Refactor the code to revert with the “OutOfBondsRead” error in case “self.len()” is less than 32.

```
48 if (offset > self.len() - 32) revert OutOfBoundsRead();
```

CVF-7 FIXED

- **Category** Suboptimal
- **Source** dynmem.sol

Description This function is very inefficient, as it copies the whole array just to pad it.

Recommendation Remove this function, and pad dynamic memory blocks when they are initially allocated.

Client Comment *Is unused anyway, removed.*

```
71 function pad_to_multiple_of_32(dynmem self) internal pure returns (
   ↪ dynmem padded) {
```

CVF-8 FIXED

- **Category** Documentation
- **Source** cdptr.sol

Description It is unclear, whether “start” and “end” pointers are inclusive or exclusive. Actually, both are inclusive, while a common practice is to treat “start” inclusively and “end” exclusively”.

Recommendation Clearly explain the behavior in a documentation comment and fix it if necessary.

Client Comment *It works as expected, end inclusive is usually wrong but here checks are always applied *after* a read so it basically means that the last read bytes where up to but not including the end but please double-check. Added a comment.*

```
46 function within_bounds(cdptr self, cdptr start, cdptr end) internal
   ↪ pure returns (bool within) {
```

CVF-9 INFO

- **Category** Overflow/Underflow
- **Source** WeirollVM.sol

Description Phantom overflow is possible here, i.e. a situation when the final calculation result would fit into the destination type, while certain intermediary calculation overflows.

Recommendation Use the “mulDiv” function.

Client Comment *Intentional, ‘mulDiv‘ is more expensive and it’s rare that you’d want to do that big of a computation anyway.*

```
139 self.set_reg_from_op(op, x.mulWad(y));
```

```
223 self.set_reg_from_op(op, x.divWad(y));
```

CVF-11 INFO

- **Category** Suboptimal
- **Source** WeirollVM.sol

Description This seems redundant, as dynmem doesn't store length in memory.

Recommendation Remove this logic or explain why it is necessary.

Client Comment *It can be useful when you want the bytes to be used for abi-encoding where it expects the length to be pushed first. This operation was used somewhat in Weiroll V1 so it was reimplemented.*

```
418 // Store length.
    mstore(raw_retdata_mem_addr, returndatasize())
```

8 Recommendations

CVF-12 INFO

- **Category** Procedural
- **Source** dynmem.sol

Recommendation Consider specifying as "^0.8.0" unless there is something special regarding this particular version. Also relevant for: cdptr.sol, WeirollVM.sol.

```
2 pragma solidity ^0.8.13;
```

CVF-13 INFO

- **Category** Bad datatype
- **Source** dynmem.sol

Description This type actually encapsulates two fields: offset and length, which is not obvious.

Recommendation Explain the structure of the type in a documentation comment.

Client Comment *This is already explained in the readme.*

```
6 type dynmem is uint256;
```

CVF-14 INFO

- **Category** Suboptimal
- **Source** dynmem.sol

Description This function is redundant, as it is basically an alias for "dynmem.wrap".

Recommendation Remove this function.

Client Comment *Technically yes but practically it allows you to use method syntax and also makes the meaning of the operation clearer.*

```
27 function unsafe_from_word(uint256 word) internal pure returns (
   ↪ dynmem mem_word) {
```

CVF-15 INFO

- **Category** Suboptimal
- **Source** dynmem.sol

Description This function is redundant, as it is basically an alias for “dynmem.onwrap”.

Recommendation Remove this function.

Client Comment *Technically yes but practically it allows you to use method syntax and also makes the meaning of the operation clearer.*

```
31 function to_word(dynmem self) internal pure returns (uint256) {
```

CVF-16 INFO

- **Category** Unclear behavior
- **Source** dynmem.sol

Description There are no range checks for the arguments.

Recommendation Add appropriate range checks. Namely check that “raw_mem_addr” fits into 224” bits and “length” fits into 32 bits.

Client Comment *That is intentional, that’s also why the method has a ‘unsafe_‘ to make it clear to the caller that there some extra invariants/properties that must be enforced for it to operate safely.*

```
35 function unsafe_from_raw_parts(uint256 raw_mem_addr, uint256 length)
   ↪  internal pure returns (dynmem value) {
```

CVF-17 INFO

- **Category** Suboptimal
- **Source** dynmem.sol

Recommendation This could be simplified as: if (end_offset > length) { unckeced { ... } }

```
65 if (end_offset <= length) return word;
unchecked {
```

CVF-18 INFO

- **Category** Readability
- **Source** dynmem.sol

Recommendation All brackets except for brackets around the subtraction are redundant.

Client Comment *At least to me the operator precedence for the operators in this expression isn't super intuitive so I like to have parenthesis to make it explicit. Will keep.*

```
67 word = word & ((~uint256(0)) << ((end_offset - length) * 8));
```

CVF-19 INFO

- **Category** Readability
- **Source** dynmem.sol

Recommendation Brackets around operations are redundant here.

Client Comment *Again the operator precedence here is not super intuitive to me here so I prefer to keep it for clarity.*

```
75 uint256 padded_length = (~uint256(0x1f)) & (length + 31);
```

CVF-20 FIXED

- **Category** Bad naming
- **Source** dynmem.sol

Description It is unclear, whether "offset" belongs to "dst" or "src".

Recommendation Rename "offset" to "dst_offset".

```
101 function copy_all_from(dynmem dst, uint256 offset, dynmem src)
      ↪ internal pure {
```

CVF-21 FIXED

- **Category** Bad datatype
- **Source** cdptr.sol

Recommendation The value "0x100000000" should be a named constant.

```
43 require(cdptr.unwrap(end) <= 0x100000000);
```

CVF-22 INFO

- **Category** Suboptimal
- **Source** cdptr.sol

Recommendation Consider using a constant value instead of a “sub” expression.

Client Comment *Sub is optimized away and IMO makes it clearer where the bit shift comes from.*

```
60 value := shr(sub(256, 32), calldataload(self))
```

```
66 value := shr(sub(256, 112), calldataload(self))
```

```
72 value := shr(sub(256, 160), calldataload(self))
```

CVF-23 FIXED

- **Category** Bad naming
- **Source** cdptr.sol

Description The name is confusing, as the function returns “dynmem”, not “bytes”.

Recommendation Use more specific name, like “read_as_dynmem”.

```
82 function read_bytes(cdptr self, uint256 length) internal pure
   ↪ returns (dynmem value) {
```

CVF-24 INFO

- **Category** Suboptimal
- **Source** cdptr.sol

Description Assembly is redundant here.

Recommendation Use plain unchecked addition.

Client Comment *It is not redundant, without the assembly a lot of verbose wrap & unwrapping would be required.*

```
93 assembly {
       added := add(self, byte_count)
   }
```

CVF-25 INFO

- **Category** Procedural
- **Source** IWETH.sol

Description This version requirement is inconsistent with other files in the same code base.

Recommendation Use consistent version requirements across code base. Also relevant for: WeirollErrors.sol, constants.sol.

Client Comment *The file could conceivably be used in other cases, it accurately describes when it'd be valid to use, I think it's fine.*

```
2 pragma solidity ^0.8.0;
```

CVF-26 INFO

- **Category** Bad naming
- **Source** IWETH.sol

Description Despite the name, this interface contains only a subset of the WETH API.

Recommendation Use more specific name or define the full API.

```
5 interface IWETH {
```

CVF-27 INFO

- **Category** Suboptimal
- **Source** WeirollErrors.sol

Recommendation These errors could be made more useful by adding certain parameters into them. Also relevant for: WeirollVM.sol

Client Comment *Yes but that'd also make the resulting bytecode larger, it could also incur added runtime gas overhead elsewhere because of the stackscheduler, will omit.*

```
6  error UnrecognizedInstruction();
   error InvalidInputByte();
   error InvalidSliceParams();
   error InvalidOutputByte();
10 error UnexpectedReturndata();
```

```
12 error CallFailed();
   error InputNotSupportedInContext();
```

```
15 error EndOfCalldata();
```

CVF-28 INFO

- **Category** Procedural
- **Source** constants.sol

Recommendation This library should be moved into a separate file named "Mask.sol".

Client Comment *It could but it's so small and tightly related to constants so I think it's fine where it is.*

```
7  library Mask {
```

CVF-29 INFO

- **Category** Bad naming
- **Source** constants.sol

Description Despite the name, this file contains not only constants, but also functions.

Recommendation Move functions into other files or rename the file.

Client Comment *The methods are related to interepreting and use the constants so I think it's fine.*

```
11 function reg_idx(uint256 input_byte) internal pure returns (uint256)
   ↪  {
```

```
15 function call_type(uint256 input_byte) internal pure returns (
   ↪ uint256) {
```

```
19 function contains(uint256 group, uint256 op) internal pure returns (
   ↪ bool c) {
```

```
87 function is_immediate_input(uint256 input_byte) internal pure
   ↪ returns (bool is_imm) {
```

```
103 function is_word_to_reg(uint256 output_byte) internal pure returns (
   ↪ bool) {
```

```
107 function word_index(uint256 output_byte) internal pure returns (
   ↪ uint256) {
```

CVF-30 INFO

- **Category** Procedural
- **Source** constants.sol

Recommendation This library should be moved into a separate file named "Op.sol".

```
26 library Op {
```

CVF-31 INFO

- **Category** Procedural
- **Source** constants.sol

Recommendation This library should be moved into a separate file named “Priority.sol”.

```
57 library Priority {
```

CVF-32 FIXED

- **Category** Unclear behavior
- **Source** constants.sol

Description The list of “utility” operations looks arbitrary. It is unclear, why the list does include “LTE” and “REVERT_IF_ZERO”, but doesn’t include “EQ” and “REVERT_IF_NON_ZERO”.

Recommendation Add more operations into the list or clearly explain, why only these operations were added.

Client Comment *This is based on some historical usage analysis that was done on Weiroll V1. Added comment.*

```
61 // Op.EXTRACT_WORD, Op.SUB_SAFE, Op.WAD_MUL, Op.LTE, Op.WETH_DEPOSIT
   ↪ , Op.REVERT_IF_ZERO, Op.HALT
uint256 internal constant UTILS = 0
   ↪ x800000ff0000000000000000000040000ff0000ff000000ff000000000000ff00
   ↪ ;
```

CVF-33 INFO

- **Category** Procedural
- **Source** constants.sol

Recommendation This library should be moved into a separate file named “CallType.sol”.

```
65 library CallType {
```

CVF-34 INFO

- **Category** Procedural
- **Source** constants.sol

Recommendation This library should be moved into a separate file named “Input.sol”.

```
71 library Input {
```

CVF-35 FIXED

- **Category** Documentation
- **Source** constants.sol

Description It is unclear how the value “0x14” was derived.

Recommendation Explain in a comment.

Client Comment *Added explanation.*

```
89 is_imm := lt(xor(input_byte, 0x14), 4)
```

CVF-36 INFO

- **Category** Procedural
- **Source** constants.sol

Recommendation This library should be moved into a separate file named “Output.sol”.

```
94 library Output {
```

CVF-37 FIXED

- **Category** Documentation
- **Source** constants.sol

Description This check treats all bytes under 0x80 as “word_to_reg”, while the value of the “WORD_TO_REG” constant is just 0z00.

Recommendation Explain the logic in a documentation comment.

Client Comment *In the readme it describes how the whole range ‘0x00-0x7f‘ is dedicated to WORD_TO_REG, ‘Output.WORD_TO_REG‘ is just the start of the range.*

```
103 function is_word_to_reg(uint256 output_byte) internal pure returns (
       ↪ bool) {
        return output_byte & 0x80 == 0;
```

CVF-38 INFO

- **Category** Procedural
- **Source** WeirollVM.sol

Description We didn’t review these files.

```
6 import {SafeTransferLib} from "solady/src/utils/SafeTransferLib.sol"
     ↪ ;
  import {FixedPointMathLib} from "solady/src/utils/FixedPointMathLib.
     ↪ sol";
```

CVF-41 INFO

- **Category** Unclear behavior
- **Source** WeirollVM.sol

Description There should be a function to extract any input except for “EndOfInputs”.

Recommendation Implement such a function and use it where appropriate.

151
```
(current, input_type, word) = self.any_input_as_word(current,
   ↪ NO_WRITE_PTR);
require(input_type != InputType.EndOfInputs,
   ↪ InputNotSupportedInContext());
```

168
```
    (current, type1, word1) = self.any_input_as_word(current,
       ↪ NO_WRITE_PTR);
```

171
```
    (current, type2, word2) = self.any_input_as_word(current,
       ↪ NO_WRITE_PTR);
```

174
```
    require(type1 != InputType.EndOfInputs,
       ↪ InputNotSupportedInContext());
    require(type2 != InputType.EndOfInputs,
       ↪ InputNotSupportedInContext());
```

CVF-42 INFO

- **Category** Suboptimal
- **Source** WeirollVM.sol

Description Input type is checked twice for each input.

Recommendation Refactor the code to check only once.

176
```
uint256 length1 = type1 == InputType.Word ? 32 : dynmem_lib.
   ↪ unsafe_from_word(word1).len();
uint256 length2 = type2 == InputType.Word ? 32 : dynmem_lib.
   ↪ unsafe_from_word(word2).len();
```

181
```
if (type1 == InputType.Word) {
```

187
```
if (type2 == InputType.Word) {
```

CVF-43 INFO

- **Category** Suboptimal
- **Source** WeirollVM.sol

Description The function “unsafe_from_word” is called twice for each input.

Recommendation Refactor the code to call only once.

176
```
uint256 length1 = type1 == InputType.Word ? 32 : dynmem_lib.
   ↪ unsafe_from_word(word1).len();
uint256 length2 = type2 == InputType.Word ? 32 : dynmem_lib.
   ↪ unsafe_from_word(word2).len();
```

185
```
    concatenated.copy_all_from(0, dynmem_lib.unsafe_from_word(word1)
       ↪ );
```

191
```
    concatenated.copy_all_from(length1, dynmem_lib.unsafe_from_word(
       ↪ word2));
```

CVF-44 INFO

- **Category** Suboptimal
- **Source** WeirollVM.sol

Description These two function have much common code.

Recommendation Refactor to reduce code duplication.

488
```
function fixed_input(WeirollVM self, cdptr current) internal view
   ↪ returns (cdptr, uint256 word) {
```

533
```
function any_input_as_word(WeirollVM self, cdptr current, uint256
   ↪ write_to_ptr)
```



ABDK
Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 40 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting